

Cross-Platform Smoke Test

What you must test on each OS — and what you can safely assume from one codebase

APRS Command · James Rospopo (KE4CON)

Companion to the full click-through (`FINAL_RELEASE_VALIDATION_CHECKLIST.md`)

The idea in one paragraph

APRS Command is one C#/.NET + Avalonia codebase, so the **logic and most of the UI are genuinely portable** — test those *once*. But a handful of features touch the operating system directly, and **those are where cross-platform apps break**. This page is *only* those OS-boundary items. Run the full click-through once (on your dev machine), then run this short smoke on each OS you intend to ship — **especially with the radio connected**, since serial is the riskiest surface.

Safe to assume portable — do NOT re-test per OS: APRS packet parsing, all math/business logic, the map and most UI (Avalonia → Skia → Mapsui render the same everywhere), and the network transports (APRS-IS, KISS-TCP, AGWPE — plain sockets). If these work on one OS, they work on all.

Before you start on Linux / Raspberry Pi (one-time setup)

Two gotchas will otherwise look like "the app is broken" when it's really the OS:

- **SERIAL** Add your user to the serial group, then log out/in: `sudo usermod -aG dialout $USER`. Without this, the radio's port won't open (permission denied) even though it's plugged in. (Some distros use `uucp` instead of `dialout`.)
- **VOICE** Install the Linux/Pi speech engine: `sudo apt install espeak-ng`. It is not preinstalled on a fresh system, so voice alerts silently do nothing until it's there.

The per-OS smoke checklist

Run each row on each OS you support. Columns: **Win** = Windows x64 · **Mac** = macOS (test one of Intel/Apple-Silicon thoroughly, spot-check the other) · **Lin** = Linux x64 · **Pi** = Raspberry Pi (linux-arm64). Tick when it works.

#	Check	Win	Mac	Lin	Pi
1	App launches and the window renders. No crash on start; no clipped/overlapping text; native file-picker dialogs open (Import GPX/KML, Export). Avalonia is portable but fonts, DPI scaling, and native dialogs differ per OS.	☐	☐	☐	☐
2	Serial port discovery. Your radio's port appears in the connection dropdown (COM3 on Windows; /dev/tty* on Linux/Pi; /dev/cu.* on macOS). Port enumeration is OS-specific code (SerialPortDiscovery).	☐	☐	☐	☐
3	Radio connects (serial open + receive). Open the KISS/serial (or CI-V) port; confirm the connection state goes to <i>Connected</i> and received packets flow in. System.IO.Ports behaves differently per OS and is historically weakest on Linux/macOS — this is the #1 thing that won't "just work" from a Windows test.	☐	☐	☐	☐
4	★ Radio TRANSMIT on a dummy load / low power. Enable RF transmit for the port, key a beacon, and confirm the TNC/rig actually transmits. The transmit-safety gates are portable code, but the physical serial write to the TNC is OS-specific. Do this per OS with hardware. Dummy load or low power only.	☐	☐	☐	☐
5	GPS over serial (if you use an external NMEA GPS). The receiver is found and a fix updates your position. Same serial layer as the radio (SerialNmeaGpsSource).	☐	☐	☐	☐
6	Voice alerts speak. Trigger a voice alert and hear it. Three different engines: System.Speech (Win), say (Mac), espeak-ng (Linux/Pi — install it first, see above).	☐	☐	☐	☐
7	Sound alerts play (chimes/beeps) and audio-device selection works. Audio playback + device enumeration have OS branches (SoundAlertService , AudioDeviceDiscovery).	☐	☐	☐	☐
8	Managed Direwolf modem launches (only if you use it). Starting the managed modem spawns Direwolf and connects via loopback KISS. Process.Start of direwolf.exe vs direwolf ; the binary must be on PATH per OS.	☐	☐	☐	☐
9	Mobile Companion / FieldCommand feed is reachable from a phone/another device on the LAN (open the shown URL; the map/data loads). HttpListener binding to a non-localhost address differs per OS (Windows can need a URL-ACL or elevation; Linux/macOS bind freely).	☐	☐	☐	☐
10	Settings & logs persist across a restart. Change a setting, restart, confirm it stuck; confirm packet/event logs are written. Data lands in a different folder per OS (GetFolderPath(ApplicationData)), and Linux filesystems are case-sensitive — a path that works on Windows can fail on Linux.	☐	☐	☐	☐
11	Pi-only: GPIO PTT (if you wire PTT to a GPIO pin). The pin toggles on transmit. GpioPttController is Raspberry-Pi-specific hardware code that only exists on that target.	—	—	—	☐

HIGHEST RISK Rows **2–5** (everything serial — the radio and GPS) are the most likely to behave differently from your Windows test. If you only have time for a few checks on a given OS, do those. The rest of the app riding on top is already portable.

How much to test where

Platform	How much
Windows (dev box)	Full click-through + full radio test. This is your baseline.
macOS (one of Intel / Apple Silicon)	This whole smoke list, radio included. Spot-check the <i>other</i> Mac arch for launch + one serial connect.
Linux x64	This whole smoke list (after the dialout + espeak-ng setup).
Raspberry Pi (ARM64)	First-class target — run the whole list <i>on the actual Pi</i> , plus row 11 (GPIO PTT) if used. Don't infer it from x86 Linux: different arch, different hardware, most likely to be missing system deps.

Turn failures into permanent fixes. Anything that fails here is a real bug — capture the OS + exact symptom and it becomes a code fix plus a regression test, same as everything else. OS-specific bugs love to come back, so a test that pins the fixed behavior is worth the few minutes.

Source: `docs/release/CROSS_PLATFORM_SMOKE_TEST.html` (regenerate the PDF via headless Chrome/Edge `--print-to-pdf`). Companion to `FINAL_RELEASE_VALIDATION_CHECKLIST.md` .